Lecture 1.11: ES6 Array Methods and Arrow Functions

Goal of this lecture

This lecture introduces modern array techniques and ES6 syntax:

- Callbacks and higher-order functions (functions passed into other functions)
- `forEach`, `filter`, `map`, `reduce`
- Checks: `every`, `some`
- Searching: `find`
- Flattening nested arrays: `flat`
- Arrow functions: `(x) => x * 2`

Functions are values in JavaScript

In JS, a function can be stored in a variable and passed as an argument. That's why `forEach` can accept a function.

```
function printNumber(num = 0) {
    console.log(num);
}
```

Output:

```
(no output — function definition only)
```


Powered by SPRK Technologies

`forEach` — loop through each element

```
var arr = [5, 2, 56, 76, 2];

arr.forEach(function(num = 0) {
    console.log(num);
});

arr.forEach(printNumber);

arr.forEach((num) => console.log(num));
```

Output:

```
5
2
56
76
2
5
2
56
76
2
5
2
56
76
2
```

Callback parameters

The callback can receive:

- current element
- index
- the whole array

Powered by SPRK Technologies

```
arr.forEach((num, idx, a) => {
    console.log(idx, ":", num);
});
```

Output:

```
0 : 5
1 : 2
2 : 56
3 : 76
4 : 2
```

Mutating the array inside `forEach`

Your lecture doubles each element by writing back into the same array:

```
arr.forEach((num, idx, a) => a[idx] = a[idx] * 2);
console.log(arr.toString());
```

Output:

```
10,4,112,152,4
```

> **Instructor insight:** Mutating arrays is sometimes okay, but many modern patterns prefer `map` to create a new array instead of changing the old one.

Powered by SPRK Technologies

`filter` — select items into a new array

```
var below10 = arr.filter(ele => ele < 10);
console.log(below10);
```

Output:

```
[4,4]
```

`reduce` — combine values into one result

`reduce` is used to compute a single final value (sum, min, max, etc.). Your lecture uses it for all three.

```
var sum = arr.reduce((sum, curr) => sum + curr, 0);

var min = arr.reduce((min, curr) => (min > curr ? curr : min), arr[0]);
var max = arr.reduce((max, curr) => (max < curr ? curr : max), arr[0]);

console.log(sum, min, max);
```

Output:

```
282 4 152
```

> **Important:** Always think about the initial value in `reduce`. For sum, `0` is perfect. For min/max, starting with `arr[0]` is common (but only if the array isn't empty).

Powered by SPRK Technologies

`map` — transform every element into a new array

```
var doubled = arr.map(ele => ele * 2);
console.log(doubled.toString());
```

Output:

```
20,8,224,304,8
```

`every`, `some`, `find`

```
console.log(arr.every(ele => ele % 2 == 1)); // all odd?
console.log(arr.some(ele => ele % 2 == 1));  // at least one odd?
console.log(arr.find(ele => ele == 56));     // returns the element (or undefined)
```

Output:

```
false
false
undefined
```

Flatten nested arrays with `flat`

```
var nested = [1, 2, [3, 4, 5, [6, 7, 8, [9, 10]]]];

console.log(nested.flat(1));
console.log(nested.flat(2));
console.log(nested.flat(Infinity));
```


Powered by SPRK Technologies

Output:

```
[1,2,3,4,5,[6,7,8,[9,10]]]
[1,2,3,4,5,6,7,8,[9,10]]
[1,2,3,4,5,6,7,8,9,10]
```

What does “depth” mean?

- `flat(1)` removes one level of nesting
- `flat(2)` removes two levels
- `flat(Infinity)` removes all nesting levels

Practice tasks

1. Given an array of marks, use `filter` to create an array of marks ≥ 50.
2. Use `map` to convert Celsius temperatures to Fahrenheit.
3. Use `reduce` to compute the average of an array of numbers.
4. Create a nested array and try `flat(1)` and `flat(2)`.

JS Refined Notes — Lecture 2.3

Powered by SPRK Technologies